



Scalable Distributed Computing - 2026/27

Lecture 1 - Foundations of Scalability in Distributed Systems

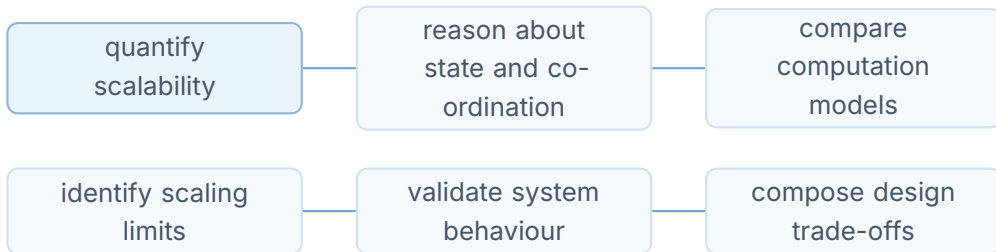
Patrizio Dazzi patrizio.dazzi@unipi.it

What this course is about

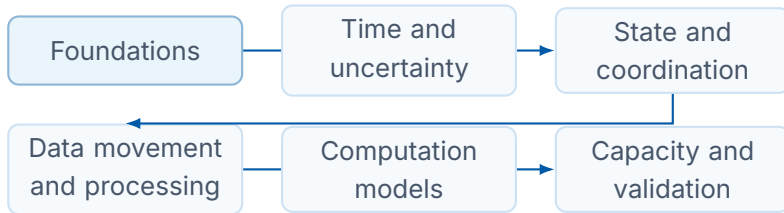
Designing, analysing and validating distributed systems whose workload, data, and resource demands change at scale.

The goal is not to memorize systems, but to understand why their design choices work or fail.

What you will learn to do



The course in six modules



A guided anthology of distributed computing



Different distributed problems require different abstractions:
replication, consensus, dataflow, actors, graph computation,
queueing and overload control.

How we will reason about systems

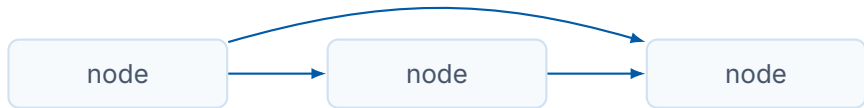


Technologies appear only when they make the model concrete.

What is a distributed system?

Working definition

A distributed system consists of autonomous computing elements that cooperate through communication so that users or applications experience a coherent system.



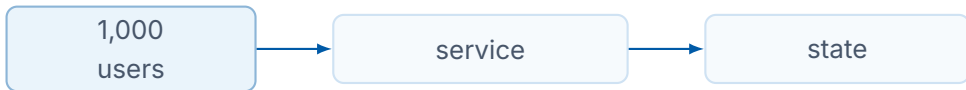
autonomy + communication + coherence

Adapted from Tanenbaum and van Steen, *Distributed Systems*, and van Steen/Tanenbaum, *Computing* 2016.

1. The scaling question

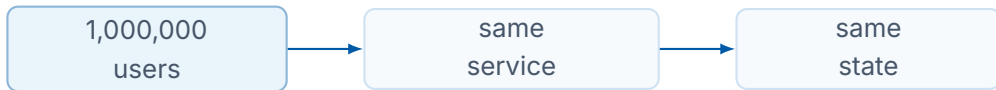
From a working system to a larger requirement

A system serving 1,000 users



The system works under the current workload and requirements.

Tomorrow: 1,000,000 users



A change of scale is a change in the system problem.

What has changed?

arrival rate

concurrency

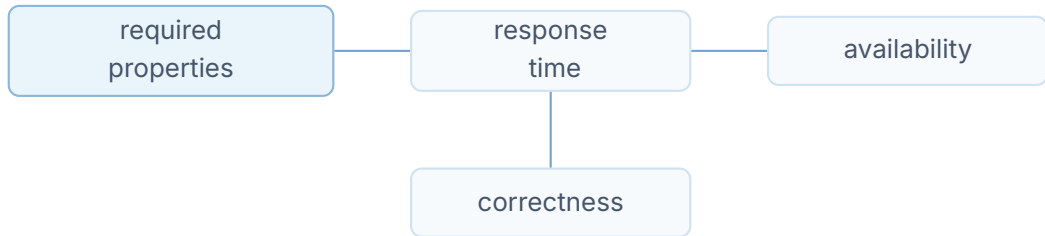
data volume

resource demand

latency pressure

operational risk

The requirements did not disappear



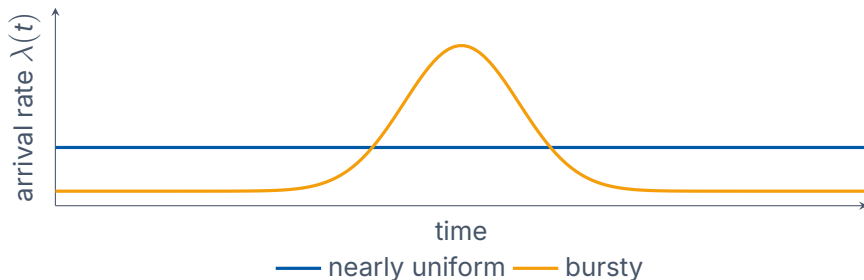
The workload grows; the expected properties remain part of the design problem.

One million users is not a workload specification



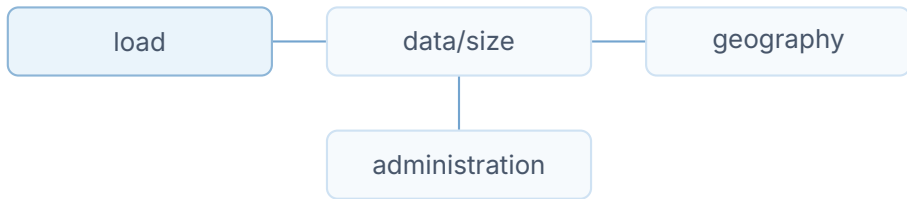
The same user count can imply very different system loads.

Same total work, different workload



Same integral over time; different instantaneous pressure on the system.

What can scale?



Scalability is multidimensional: the dimension that grows must be stated explicitly.

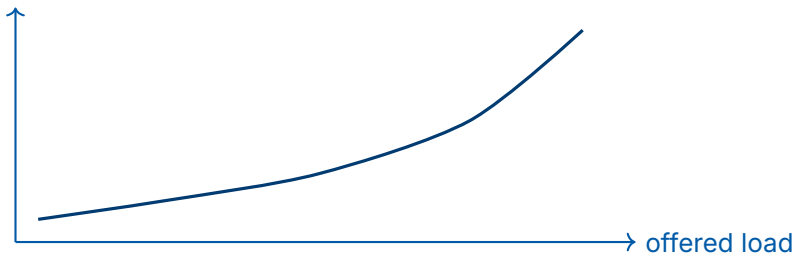
Bondi, WOSP 2000; van Steen and Tanenbaum, Computing 2016.

Load scalability

Bondi's perspective

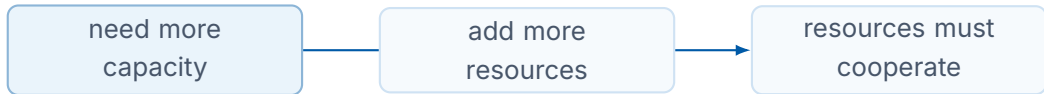
Load scalability concerns whether a system can operate gracefully under heavier or lighter loads, avoiding undue delay, unproductive resource consumption, and contention.

delay / contention / resource usage



Bondi, Characteristics of scalability and their impact on performance, WOSP 2000.

Why this is a distributed-systems lecture



When additional capacity is obtained through cooperating nodes, scalability becomes a distributed-system problem.

2. Quantifying scaling

Finite workloads: strong/weak scaling, speedup, efficiency

Two scaling questions

Finite workload

Complete this computation.
How does completion time
change as resources grow?

Continuous service

Keep serving incoming work.
How do throughput, latency
and concurrency relate?

Strong scaling

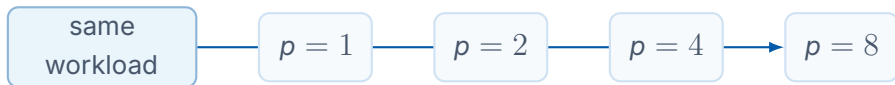
Fixed total workload, increasing resources.

$$W_{\text{total}} = \text{constant}, \quad p \uparrow$$

Question: how much faster can the same work be completed?

Standard terminology in parallel performance analysis; see NERSC performance documentation.

Strong scaling as an experiment



Measure completion time for the same workload under increasing parallel resources.

Weak scaling

Fixed work per processing element, increasing resources and workload.

$$\frac{W(p)}{p} = \text{constant}, \quad p \uparrow$$

Question: how much more work can be handled while keeping per-resource pressure comparable?

Weak scaling as an experiment



The workload grows with the amount of available parallel resources.

Same system, two different questions

Strong scaling

How much faster can the same work be completed?

Weak scaling

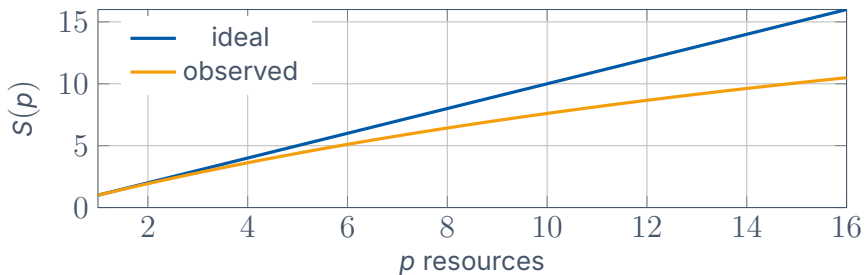
How much more work can be handled with proportionally more resources?

Speedup

$$S(p) = \frac{T(1)}{T(p)}$$

Same workload. Compare completion time on one resource with completion time on p resources.

Ideal and observed speedup



Efficiency

$$E(p) = \frac{S(p)}{p}$$

Efficiency measures how much of the ideal linear speedup is retained when resources are added.

A small scaling experiment

toprule p	$T(p)$ seconds	$S(p) = T(1)/T(p)$	$E(p) = S(p)/p$
1	100.0	1.00	1.00
2	54.0	1.85	0.93
4	31.0	3.23	0.81
8	21.0	4.76	0.60
16	17.0	5.88	0.37

More resources

help, but not linearly.

3. A first limit: Amdahl

Fixed workload and non-parallelizable work

Why does speedup depart from the ideal?



A fixed workload may contain work that cannot benefit from additional parallel resources.

A fixed workload with two components



serial

parallelizable

s is the fraction of

execution time that remains serial in the model.

Amdahl's Law

Model assumption

For a fixed workload, a fraction s of execution time is non-parallelizable, while the remaining fraction $1 - s$ is perfectly parallelized across p resources.

The law gives the resulting speedup under these assumptions.

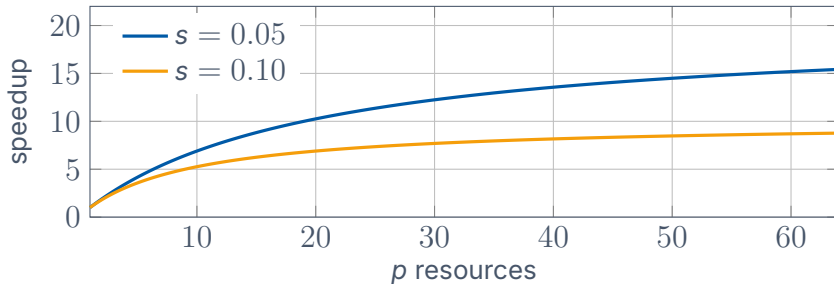
Amdahl, Validity of the single processor approach to achieving large scale computing capabilities, AFIPS 1967.

Amdahl's Law: formula

$$S_A(p) = \frac{1}{s + \frac{1-s}{p}}$$

The non-parallelizable fraction remains present regardless of how many resources are added.

Amdahl as a curve



Worked example: $s = 0.05$

toprule p	$S_A(p)$	interpretation
4	$\frac{1}{0.05+0.95/4} = 3.48$	useful, not ideal
16	$\frac{1}{0.05+0.95/16} = 9.14$	diminishing returns
64	$\frac{1}{0.05+0.95/64} = 15.42$	near the asymptote

The asymptotic limit

$$\lim_{p \rightarrow \infty} S_A(p) = \frac{1}{s}$$

If $s = 0.05$, even infinitely many resources cannot exceed
 $20\times$ speedup in this model.

What Amdahl models - and what it abstracts away

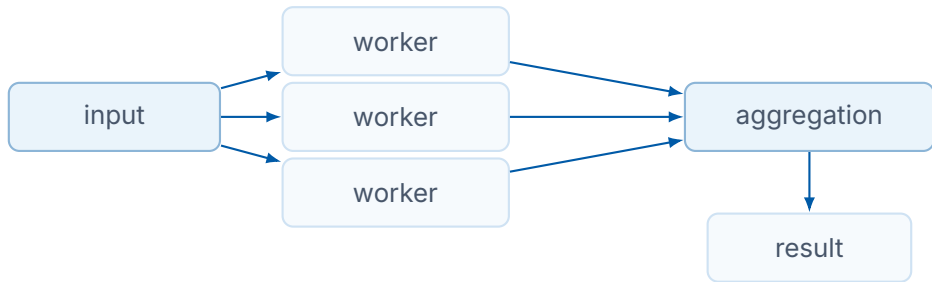
Inside the model

A fixed workload with a non-parallelizable fraction s and an ideally parallelizable fraction $1 - s$.

Not explicit in the model

Communication overhead, synchronization cost, load imbalance, cache effects, scheduling overhead.

A synthetic distributed example



If preparation and final aggregation account for 5% of the fixed execution time, Amdahl bounds the possible speedup.

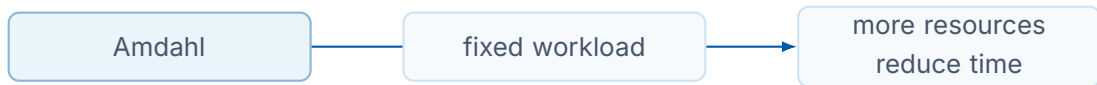
The lesson from Amdahl

To improve strong scaling, adding resources is not enough: the non-scalable part of the workload must be reduced or removed.

4. Changing the question

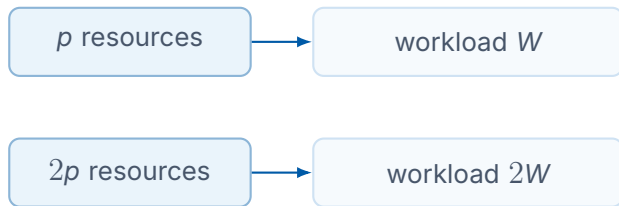
Gustafson: scaled workloads instead of fixed workloads

Must the workload stay fixed?



Large systems are often used to handle larger workloads, not only to finish the same workload faster.

Scaled-size computing



Increase the problem size together with the number of resources.

Gustafson's Law

Scaled workload viewpoint

Measure the serial fraction with respect to the parallel execution, then estimate how much work a serial machine would need to perform the scaled problem.

This responds to the fixed-size assumption behind Amdahl's model.

Gustafson, Reevaluating Amdahl's Law, Communications of the ACM, 1988.

Gustafson's Law: formula

$$S_G(p) = p - s(p - 1)$$

Here s is the serial fraction of the parallel execution time in the scaled-size view.

Worked example

$$s_G(64) = 64 - 0.05(64 - 1) = 60.85$$

With scaled-size interpretation, the same numerical serial fraction leads to a different question and a different conclusion.

Amdahl and Gustafson ask different questions

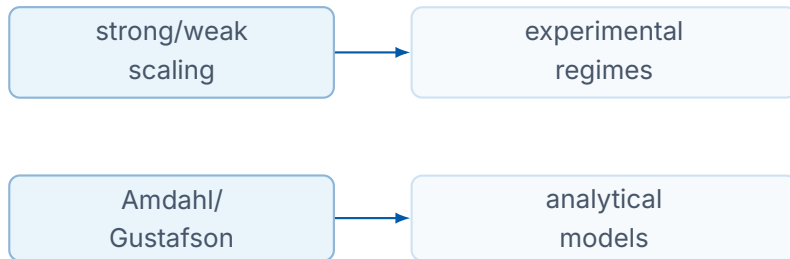
Amdahl

Fixed workload.
How much faster can the
same work be completed?

Gustafson

Scaled workload.
How much larger a problem
can be handled in a compa-
rable execution envelope?

Strong/weak scaling and Amdahl/Gustafson



They are closely related, but they are not synonyms.

5. From jobs to services: Little's Law

Throughput, latency, and work in flight

From finite jobs to continuous services

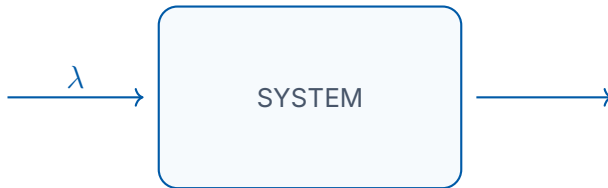
Finite job

There is a defined computation to complete. Completion time is central.

Continuous service

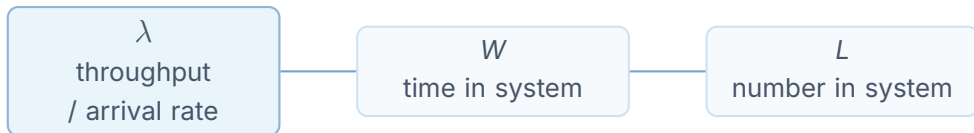
Work keeps arriving. Throughput, response time and concurrency become central.

First, choose a system boundary



the boundary defines what is counted in L and W

Three average quantities



Little's Law relates their long-run averages under suitable stability assumptions.

Little's Law

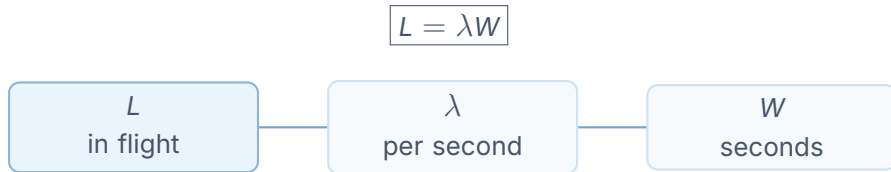
Statement

Under appropriate stationarity and finiteness conditions, the long-run average number of items in a system equals arrival rate times average time spent in the system.

$$L = \lambda W$$

Little, A proof for the queuing formula: $L = \lambda W$, Operations Research, 1961.

Little's Law: interpretation



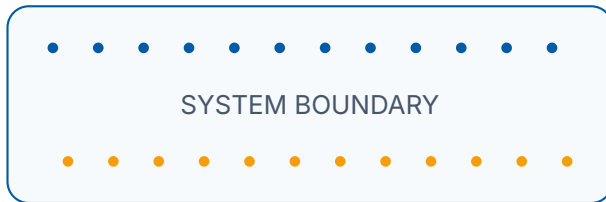
Worked example

$$\lambda = 1000 \text{ req/s}, \quad W = 0.2 \text{ s}$$

$$L = \lambda W = 1000 \cdot 0.2 = 200$$

On average, 200 requests are inside the chosen boundary.

200 requests in flight



dots are symbolic: the average count is $L = 200$

If response time doubles

$$\lambda = 1000 \text{ req/s}, \quad W = 0.4 \text{ s}$$

$$L = \lambda W = 400$$

At the same throughput, twice the response time implies
twice the average work in flight.

What Little's Law does not tell us

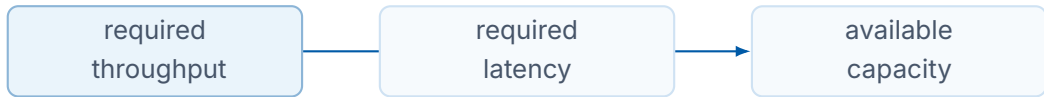
Little's Law relates averages; it does not determine the response time W , the capacity limit, or the effect of burstiness.

Those questions lead to queueing, overload and capacity analysis later in the course.

6. From capacity to distribution

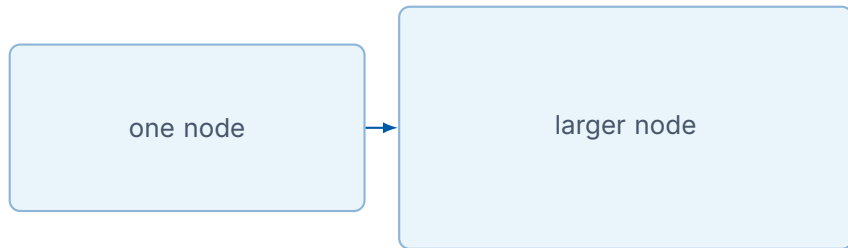
Scale-up, scale-out, and what changes

When the current configuration is no longer enough



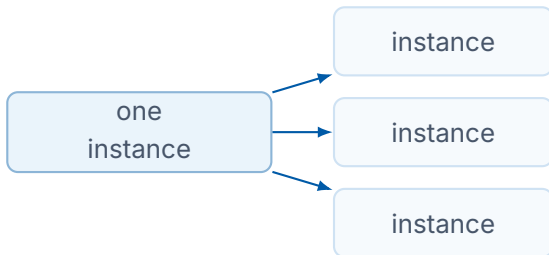
If the required behaviour cannot be achieved, the resource configuration must change.

Scale up



Add resources to an existing node or system instance.

Scale out



Add more nodes or instances.

Two ways of adding capacity

Scale up

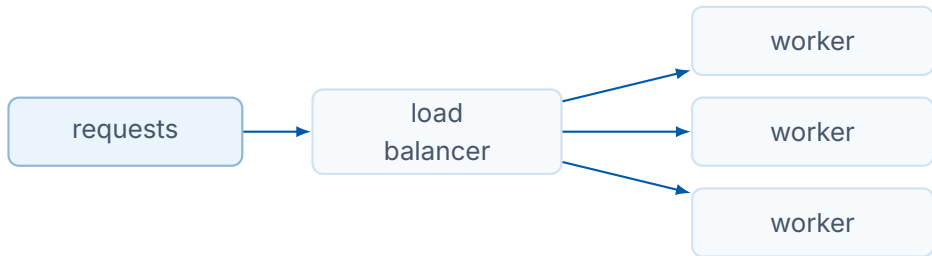
More capability in a single system instance. Does not by itself create a distributed system.

Scale out

More cooperating instances. This commonly introduces distributed-system concerns.

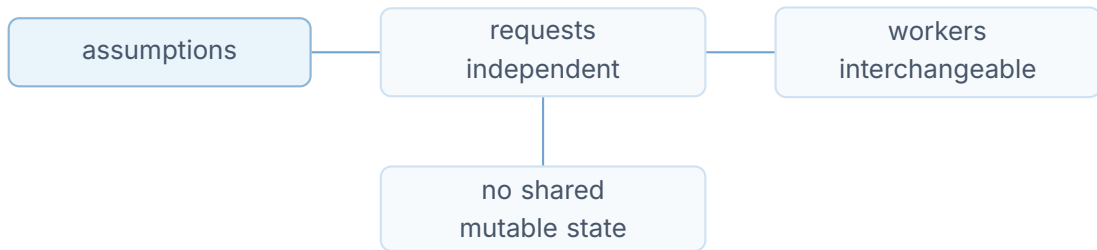
See common cloud/system engineering terminology for scale-up vs scale-out; e.g., IBM Think overview.

``Just add a load balancer''



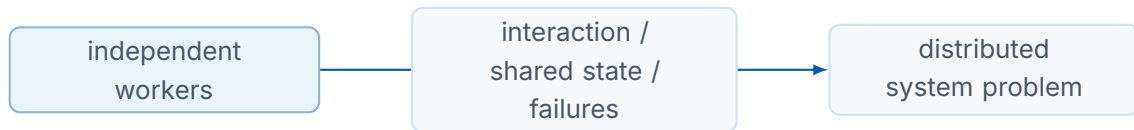
This is a useful first approximation only under favourable assumptions.

The easy case: independent, stateless requests



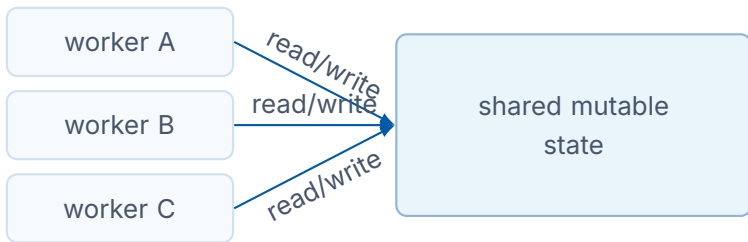
Under these assumptions, horizontal replication can increase capacity until another resource becomes limiting.

Capacity scaling relies on assumptions



The rest of the course studies what happens when these assumptions do not hold.

Add shared mutable state



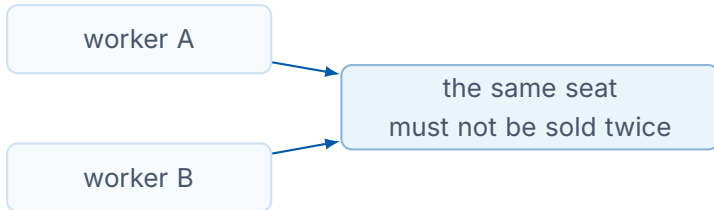
Scaling workers is no longer the same as scaling access to state.

Add remote interaction



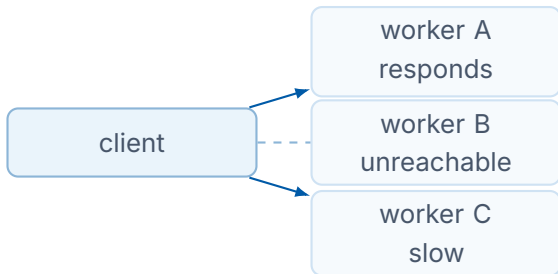
Communication becomes part of the execution path.

Add a distributed invariant



When multiple nodes must preserve a shared property, coordination enters the design problem.

Add partial failure



In distributed systems, parts may fail or become unreachable while others continue operating.

Partial failure is a standard distinguishing feature in distributed-systems texts; see van Steen/Tanenbaum.

7. Wrap-up

What we know now

What we established today

1. A scalability claim requires an explicit workload and resource model.
2. Fixed-size and scaled-size experiments answer different questions.
3. Amdahl and Gustafson expose different consequences of those choices.
4. Little's Law links throughput, response time and work in flight.
5. Scale-out turns a capacity problem into a distributed-system problem when nodes must interact, share state, preserve invariants or fail independently.

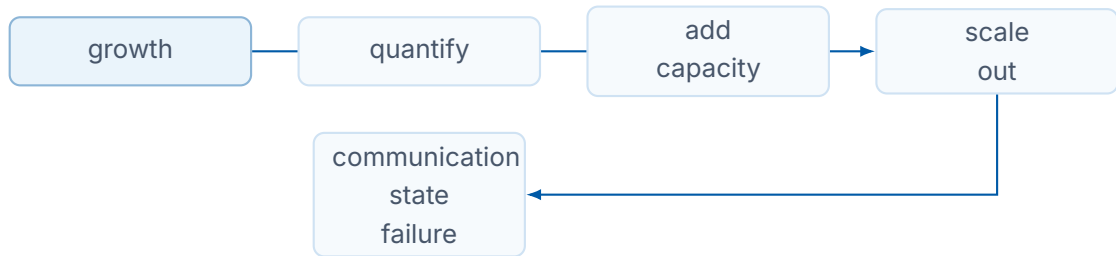
The quantitative toolkit we now have

$$S(p) = \frac{T(1)}{T(p)} \quad E(p) = \frac{S(p)}{p}$$

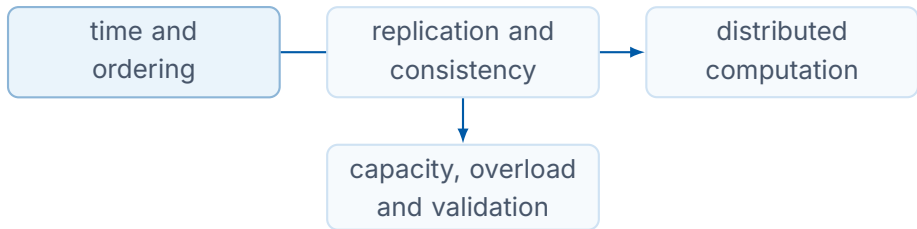
$$S_A(p) = \frac{1}{s + \frac{1-s}{p}} \quad S_G(p) = p - s(p-1)$$

$$L = \lambda W$$

From scalability to distributed scalability



What the course builds on top of this



Each later topic changes one or more terms of the scalability problem.

Next lecture

Where do scalability constraints come from?

Decomposition, boundaries, state, control, interaction and temporal assumptions.

References

Primary sources used in this lecture

References I

- G. M. Amdahl. *Validity of the single processor approach to achieving large scale computing capabilities*. AFIPS Conference Proceedings, 1967.
- J. L. Gustafson. *Reevaluating Amdahl's Law*. Communications of the ACM, 31(5), 1988.
- J. D. C. Little. *A proof for the queuing formula: $L = \lambda W$* . Operations Research, 9(3), 1961.
- A. B. Bondi. *Characteristics of scalability and their impact on performance*. WOSP, 2000.

References II

- M. van Steen and A. S. Tanenbaum. *A brief introduction to distributed systems*. Computing, 98, 2016.
- A. S. Tanenbaum and M. van Steen. *Distributed Systems*. 4th edition, 2024.
- NERSC Documentation. *Parallelism: strong and weak scaling*. Performance documentation.
- IBM Think. *Scale up vs. scale out*. Overview article.